

### Overview:

The last lab for the first module is strictly a demonstration of generating Mandelbrot using the CPU, and a second block using PyCUDA to generate Mandelbrot on the GPU. The scripts output the time difference in producing the Mandelbrot graphs.

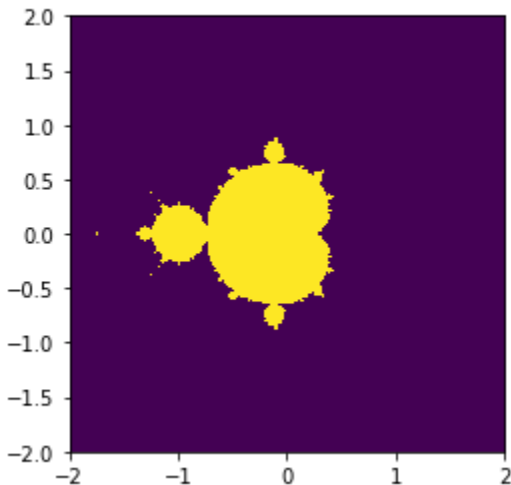
### CPU Implementation:

The CPU approach relies on nested loops. Because it processes one pixel at a time, the performance is slowed by the overhead of the Python interpreter.

- 1) **Processing:** The processing of escape time for pixel (x,y) is sequential and the CPU must calculate this before moving to (x + 1, y)
- 2) **Bottleneck:** Python's lack of native vectorization means the hardware is significantly constrained.
- 3) **Performance:** In this test it took a massive 120.19 seconds to calculate the graph, and 0.33 seconds to create the image.

It took 120.19067215919495 seconds to calculate the Mandelbrot graph.

It took 0.3307960033416748 seconds to dump the image.



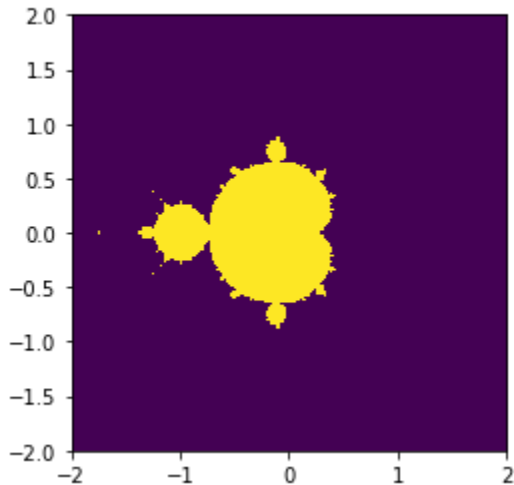
### GPU Implementation:

The GPU implementation shifts the workload to thousands of threads. Using PyCUDA, the Mandelbrot set is calculated by treating every point in the lattice as an independent thread.

- 1) **Processing:** The ElementwiseKernel allows the GPU to compute the escape values for thousands of pixels in parallel.

- 2) **Data Transfer:** This method introduces the overhead of data transfer which becomes negligible when working with large inputs.
- 3) **Performance:** despite the overhead the calculation took 2.79 seconds to calculate the graph, and 0.12 seconds to create the image.

It took 2.795729637145996 seconds to calculate the Mandelbrot graph.  
It took 0.12658262252807617 seconds to dump the image.



### Observations:

Based on the time it took to generate the graph, it is very obvious that for large inputs the GPU outperforms the CPU by an incredibly significant degree. Specifically a 43x speedup going from sequential to parallel processing. This serves as a good demonstration as many of the previous demonstrations in this module show marginal gains that can sometimes be dwarfed by the overhead of memory transfer, however in this case that transfer is incredibly insignificant compared to the performance increase.